

Лабораторная работа

Номер 1

Жаворонков Кирилл Александрович

Цель работы

Основной целью работы является установка и настройка языка программирования Julia и среды интерактивной разработки JupyterLab, а также изучение базовых конструкций языка Julia: типов данных, преобразования типов, функций, векторов, матриц и операций над ними.

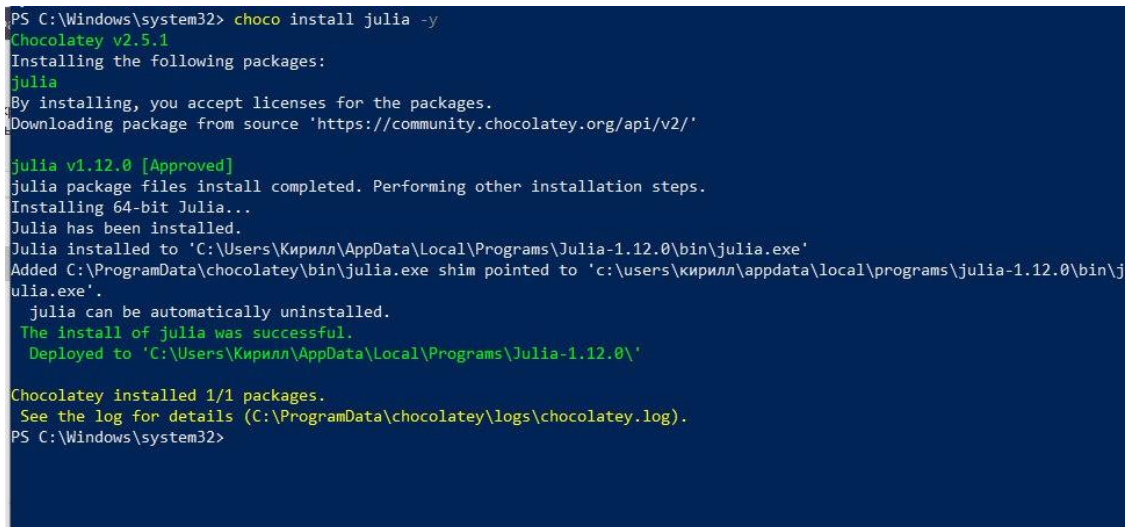
Выполнение лабораторной работы

1. Установка Julia

Для выполнения лабораторной работы использовалась операционная система Windows. Установка Julia выполнялась с помощью менеджера пакетов Chocolatey. В командной строке была введена команда:

```
choco install julia -y
```

В результате была установлена Julia версии 1.12.0. Установщик сообщил об успешном завершении установки и разместил исполняемый файл Julia в каталоге пользователя (рис. 1).



```
PS C:\Windows\system32> choco install julia -y
Chocolatey v2.5.1
Installing the following packages:
julia
By installing, you accept licenses for the packages.
Downloading package from source 'https://community.chocolatey.org/api/v2/'
julia v1.12.0 [Approved]
julia package files install completed. Performing other installation steps.
Installing 64-bit Julia...
Julia has been installed.
Julia installed to 'C:\Users\Кирилл\AppData\Local\Programs\Julia-1.12.0\bin\julia.exe'
Added C:\ProgramData\chocolatey\bin\julia.exe shim pointed to 'c:\users\кирилл\appdata\local\programs\julia-1.12.0\bin\julia.exe'.
julia can be automatically uninstalled.
The install of julia was successful.
Deployed to 'C:\Users\Кирилл\AppData\Local\Programs\Julia-1.12.0\'
Chocolatey installed 1/1 packages.
See the log for details (C:\ProgramData\chocolatey\logs\chocolatey.log).
PS C:\Windows\system32>
```

Figure 1: Установка Julia с помощью Chocolatey

2. Запуск Julia и проверка работы REPL

После установки Julia была запущена в режиме REPL. В открывшемся окне отображается версия Julia 1.12.0. Для проверки выполнения программного кода было вычислено выражение:

```
2 + 2
```

Результат выполнения равен 4, что подтверждает корректную установку и запуск Julia (рис. 2).

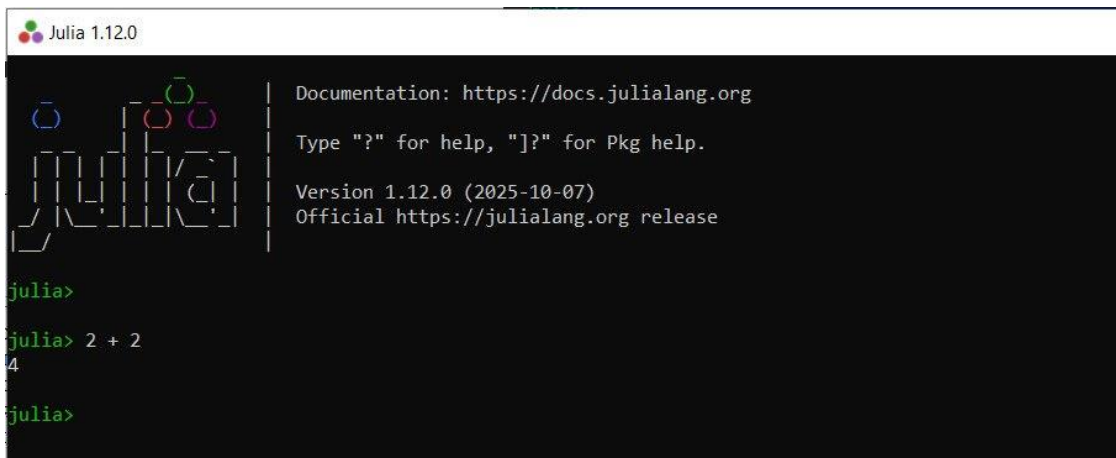


Figure 2: Запуск Julia REPL и выполнение выражения

3. Работа в JupyterLab и справочная система Julia

В JupyterLab был создан ноутбук с ядром Julia. В первой ячейке выполнено простое выражение:

```
2 + 3
```

Результат выполнения равен 5.

Также была проверена встроенная справочная система Julia. Для получения информации о функции `println` перед её именем был установлен знак вопроса:

```
?println
```

Jupyter вывел описание функции, её сигнатуру и пример использования (рис. 3).

```
[1]: 2 + 3
[1]: 5
[4]: ?println
      search: println print sprint pointer printstyled
[4]:
      println([io::IO], xs...)

Print (using print ) xs to io followed by a newline. If io is not supplied, prints to the default
output stream stdout .

See also printstyled to add colors etc.

Examples

julia> println("Hello, world")
Hello, world

julia> io = IOBuffer();

julia> println(io, "Hello", ',', " world.")

julia> String(take!(io))
"Hello, world.\n"
```

Figure 3: Выполнение кода и получение справки по println

4. Определение типов числовых величин

Для определения типа числовых значений была использована функция `typeof()`:

```
typeof(3)
```

Результатом является тип `Int64`.

Дополнительно были исследованы типы вещественного, комплексного и иррационального чисел:

```
typeof(3.5), typeof(3 / 3.5), typeof(sqrt(3 + 4im)), typeof(pi)
```

В результате были получены типы `Float64`, `Float64`, `ComplexF64` и `Irrational`. Также были рассмотрены специальные значения, возникающие при делении на ноль:

```
1.0 / 0.0
1.0 / (-0.0)
```

Результатами являются `Inf` и `-Inf` (рис. 4).

```

[6]: typeof(3)
[6]: Int64
[8]: typeof(3.5), typeof(3/3.55), typeof(sqrt(3+4im)), typeof(pi)
[8]: (Float64, Float64, ComplexF64, Irrational{π})
[9]: 1.0/0.0, 1.0/(-0.0)
[9]: (Inf, -Inf)
[11]: Int64(2.0), Char(2), typeof(Char(2))
[11]: (2, '\x02', Char)

```

Figure 4: Определение типов и специальные значения

В той же работе были проверены преобразования типов:

```
Int64(2.0), Char(2), typeof(Char(2))
```

Число 2.0 было преобразовано в Int64, а число 2 — в символ. В Julia символ отображается в виде специальной записи, соответствующей коду символа.

5. Определение функций

Функция возведения числа в квадрат была определена с помощью конструкции `function ... end`:

```

function f(x)
    x^2
end

println(f(4))

```

Результатом является число 16. После этого функция была вызвана повторно:

```
f(4)
```

Результат также равен 16 (рис. 5).

```

[18]: function f(x)
        x^2
      end
      println(f(4))
      16
[19]: f(4)
[19]: 16

```

Figure 5: Определение и вызов функции

6. Векторы и матрицы

Были созданы вектор-строка и вектор-столбец:

```
a = [4 7 6]
b = [1, 2, 3]

a[2], b[2]
```

Результатом является кортеж (7, 2), поскольку индексация в Julia начинается с единицы, а вторые элементы векторов равны 7 и 2.

Затем была создана матрица:

```
a = 1; b = 2; c = 3; d = 4;
Am = [a b; c d]
```

Получена матрица размера 2×2:

```
1 2
3 4
```

Также были выполнены операции над массивами:

```
aa = [1 2]
AA = [1 2; 3 4]
aa * AA * aa'
```

Результатом матричного выражения является матрица размера 1×1 со значением 27. Символ ' используется для транспонирования (рис. 6).

```

[20]: a = [4 7 6]
      b = [1, 2, 3]
      a[2], b[2]

[20]: (7, 2)

[22]: a = 1; b = 2; c = 3; d = 4;
      Am = [a b; c d]

[22]: 2x2 Matrix{Int64}:
      1  2
      3  4

[24]: aa = [1 2]
      AA = [1 2; 3 4]
      aa*AA*aa'

[24]: 1x1 Matrix{Int64}:
      27

[25]: aa, AA, aa'

[25]: ([1 2], [1 2; 3 4], [1; 2; 1])

```

Figure 6: Работа с векторами, матрицами и транспонированием

Самостоятельная работа

7. Функции вывода информации

Для изучения функций вывода были использованы `print()` и `println()`:

```

print("1")
print("2")

```

Результат имеет вид 12, поскольку `print()` не добавляет перевод строки.

При использовании `println()`:

```

println("1")
println("2")

```

каждое значение выводится с новой строки. Таким образом, `println()` автоматически добавляет перевод строки после вывода (рис. 7).

```
## Задания для самостоятельной работы

[26]: print("1")
      print("2")
      12

[28]: println("1")
      println("2")
      1
      2
```

Figure 7: Сравнение функций `print` и `println`

8. Функции `show()`, `readline()` и `readlines()`

Функция `show()` была проверена на векторе:

```
show([1, 2, 3])
println()
```

Она вывела текстовое представление объекта `[1, 2, 3]`.

Для чтения одной строки использовалась функция `readline()`:

```
name = readline()
println(name)
```

В ячейку был введён текст, после чего он был выведен на экран.

Также была проверена функция `readlines()`:

```
lines = readlines()
println(length(lines))
```

В данном запуске было считано нулевое количество строк, поэтому результатом `length(lines)` стало значение `0` (рис. 8).

```
[30]: show([1, 2, 3])
      println()
      [1, 2, 3]

[31]: name = readline()
      println(name)
      stdin> sdsdsd
      sdsdsd

[34]: lines = readlines()
      println(length(lines))
      0
```

Figure 8: Использование `show`, `readline` и `readlines`

9. Запись и чтение текстового файла

Для записи данных в файл была использована функция `write()`:

```
write("12.txt", "123123")
```

Функция вернула число 6, то есть количество записанных символов.

Для чтения файла применена функция `read()` с указанием типа результата `String`:

```
text = read("12.txt", String)
println(text)
```

На экран было выведено содержимое файла 123123 (рис. 9).



```
[36]: write("12.txt", "123123")

[36]: 6

[37]: text = read("12.txt", String)
      println(text)
      123123

[38]: using DelimitedFiles

[39]: write("numbers.txt", "1 2 3\n4 5 6\n7 8 9")

[39]: 17

[40]: data = readlm("numbers.txt")
```

Figure 9: Запись и чтение текстового файла

10. Чтение табличных данных с помощью `readlm()`

Для работы с функцией `readlm()` был подключён модуль `DelimitedFiles`:

```
using DelimitedFiles
```

Затем был создан текстовый файл с табличными данными:

```
write("numbers.txt", "1 2 3\n4 5 6\n7 8 9")
```

Файл был прочитан командой:

```
data = readlm("numbers.txt")
println(data)
```

В результате была получена матрица размера 3×3, содержащая числа от 1 до 9 (рис. 10).

```
[39]: write("numbers.txt", "1 2 3\n4 5 6\n7 8 9")
```

```
[39]: 17
```

```
[40]: data = readdlm("numbers.txt")
```

```
[40]: 3x3 Matrix{Float64}:  
 1.0  2.0  3.0  
 4.0  5.0  6.0  
 7.0  8.0  9.0
```

```
[41]: println(data)
```

```
[1.0 2.0 3.0; 4.0 5.0 6.0; 7.0 8.0 9.0]
```

Figure 10: Чтение табличных данных функцией `readdlm`

11. Логические операции и целочисленное деление

Были заданы логические переменные:

```
p = true  
q = false
```

Затем выполнены логические операции:

```
println(p && q)  
println(p || q)  
println(!p)
```

Результаты показали работу логических операторов `&&`, `||` и `!`.

Также были проверены функции и операторы целочисленного деления:

```
println(div(10, 3))  
println(10 % 3)
```

Функция `div(10, 3)` возвращает целую часть частного — 3, а оператор `%` возвращает остаток от деления — 1 (рис. 11).

```
## 2

[43]: x = parse{Int, "123"}
      y = parse{Float64, "1.23"}
      println(x)
      println(typeof(x))
      println(y)
      println(typeof(y))

123
Int64
1.23
Float64

[44]: number = parse{Int, readline()}
      println(number^2)

stdin> 5
25
```

Figure 11: Логические операции и целочисленное деление

12. Операции над векторами

Были созданы два вектора:

```
v1 = [1, 2, 3]
v2 = [4, 5, 6]
```

Над ними выполнены следующие операции:

```
println(v1 + v2)
println(v1 - v2)
println(v1 * 2)
```

Получены результаты:

```
[5, 7, 9]
[-3, -3, -3]
[2, 4, 6]
```

Таким образом, Julia поддерживает сложение и вычитание векторов одинакового размера, а также умножение вектора на скаляр (рис. 12).

```
## 3

[45]: a = 5
      b = 12

[45]: 12

[48]: println(a + b)
      println(a - b)
      println(a * b)
      println(a / b)
      println(a^b)
      println(sqrt(a))

      17
      -7
      60
      0.4166666666666667
      244140625
      2.23606797749979

[49]: println(a > b)
      println(a < b)
      println(a == b)
      println(a != b)
      println(a >= b)
      println(a <= b)

      false
      true
      false
      true
      false
      true

[51]: p = true
      q = false

[51]: false
```

Figure 12: Сложение, вычитание и умножение векторов на скаляр

13. Скалярное произведение и транспонирование

Для выполнения операций линейной алгебры был подключён модуль LinearAlgebra:

```
using LinearAlgebra
```

Скалярное произведение вычислено командой:

```
println(dot(v1, v2))
```

Результат равен 32:

$$1 \cdot 4 + 2 \cdot 5 + 3 \cdot 6 = 32$$

Также было выполнено транспонирование:

```
println(v1')
```

Первоначально при вызове `dot(v1, v2)` возникла ошибка `UndefVarError: dot not defined in Main`. Она была исправлена подключением модуля `LinearAlgebra`, после чего функция `dot()` стала доступна (рис. 13).

```
[51]: p = true
      q = false

[51]: false

[52]: println( p && q)
      false

[53]: println(p || q)
      println(!p)
      true
      false

[54]: println(div(10, 3))
      3

[55]: println(10 % 3)
      1
```

Figure 13: Скалярное произведение и транспонирование

14. Операции над матрицами

Были заданы матрицы:

```
A = [1 2; 3 4]
B = [5 6; 7 8]
```

Затем выполнены сложение, вычитание, матричное умножение, умножение на скаляр и транспонирование:

```
println(A + B)
println(A - B)
println(A * B)
println(2 * A)
println(A')
```

Результаты:

```
A + B = [6 8; 10 12]
A - B = [-4 -4; -4 -4]
A * B = [19 22; 43 50]
2 * A = [2 4; 6 8]
A' = [1 3; 2 4]
```

Также была продемонстрирована разница между матричным и поэлементным умножением:

```
println(A * B)
println(A .* B)
println(A .^ 2)
```

Оператор `*` выполняет матричное умножение, `.*` — поэлементное умножение, а `.^` — поэлементное возведение в степень (рис. 14).

```
## 4
[56]: v1 = [1, 2, 3]
      v2 = [4, 5, 6]

[56]: 3-element Vector{Int64}:
      4
      5
      6

[57]: println(v1 + v2)
      println(v1 - v2)
      println(v1 * 2)

      [5, 7, 9]
      [-3, -3, -3]
      [2, 4, 6]
```

Figure 14: Операции над матрицами

15. Функция `parse()`

Функция `parse()` используется для преобразования строкового представления значения в объект указанного типа:

```
x = parse{Int, "123"}
y = parse{Float64, "1.23"}

println(x)
println(typeof(x))
println(y)
println(typeof(y))
```

В результате строка `"123"` была преобразована в значение типа `Int64`, а строка `"1.23"` — в значение типа `Float64`.

Также была выполнена совместная работа `parse()` и `readline()`:

```
number = parse{Int, readline()}
println(number^2)
```

При вводе числа 5 программа вывела результат 25 (рис. 15).

```

[59]: using LinearAlgebra

[60]: println(dot(v1, v2))

      32

[61]: println(v1')

      adjoint([1, 2, 3])

```

Figure 15: Преобразование строк функцией `parse`

16. Арифметические операции и сравнение

Были заданы переменные:

```

a = 5
b = 12

```

Выполнены основные арифметические операции:

```

println(a + b)
println(a - b)
println(a * b)
println(a / b)
println(a^b)
println(sqrt(a))

```

Результаты включают сумму 17, разность -7, произведение 60, частное 0.4166666666666667, значение a^b и квадратный корень из a .

Затем были выполнены операции сравнения:

```

println(a > b)
println(a < b)
println(a == b)
println(a != b)
println(a >= b)
println(a <= b)

```

Результаты имеют логический тип и принимают значения `true` или `false` (рис. 16).

```
[62]: A = [1 2; 3 4]
      B = [5 6; 7 8]

[62]: 2x2 Matrix{Int64}:
      5  6
      7  8

[63]: println(A + B)
      println(A - B)
      println(A * B)
      println(A * 2)
      println(A')

      [6 8; 10 12]
      [-4 -4; -4 -4]
      [19 22; 43 50]
      [2 4; 6 8]
      [1 3; 2 4]

[64]: println(A * B)
      println(A .* B)

      [19 22; 43 50]
      [5 12; 21 32]

[65]: println(A .^ 2)

      [1 4; 9 16]

[ ]:
```

Figure 16: Арифметические операции и операции сравнения

Выводы

В ходе лабораторной работы была установлена и настроена среда программирования Julia в операционной системе Windows. Julia была установлена с помощью Chocolatey, после чего была проверена её работа в режиме REPL. Для интерактивного выполнения программ использовался JupyterLab с ядром Julia.

В процессе выполнения работы были изучены функция `typeof()` для определения типов данных, специальные значения `Inf` и `-Inf`, преобразования типов, определение функций, создание векторов и матриц, а также обращение к элементам массивов.

В самостоятельной части были рассмотрены функции `print()`, `println()`, `show()`, `readline()`, `readlines()`, `read()`, `write()` и `readdlm()`. С помощью `parse()` были выполнены преобразования строк в целые и вещественные числа.

Также были выполнены арифметические, логические и матричные операции, включая сложение, вычитание, умножение, деление, возведение в степень, извлечение корня, сравнение, транспонирование, поэлементные операции и

скалярное произведение. Возникшая ошибка при использовании `dot()` была исправлена подключением модуля `LinearAlgebra`.